Day 7 – Token Authentication & Interceptors

🏷️ Setup & Prerequisites

1. **Setup Checklist**:
   - **Dev Server Running**: Ensure `ng serve` is active in the terminal.
   - **Backend API Running**: Verify Django is running locally with authentication configured.
   - **IDE Ready**: Open VS Code, prepared to write an interceptor script and route guards.
2. **Prerequisites**:
   - You must have completed Day 6 successfully (real HTTP connections established and CORS authorized).

🎯 Learning Objectives

By the end of this class, you will be able to:

- Explain token-based authentication workflows in client-side applications.
- Store, retrieve, and remove authentication tokens using browser `localStorage`.
- Create and configure an HTTP Interceptor to append authentication headers to outbound requests automatically.
- Write a functional Route Guard (`CanActivateFn`) to restrict page access to authenticated users.
- Connect route guards to the Angular routing configuration.

⏱️ Classroom Timeline (45 min)

Segment	Duration	Focus
1. Hook & Big Picture	7 min	Define Client Auth flows. Trace token lifecycles from storage to requests to guards.
2. Key Concepts & Core Ideas	7 min	Explain LocalStorage persistence, HttpInterceptors, authorization headers, and router guards.

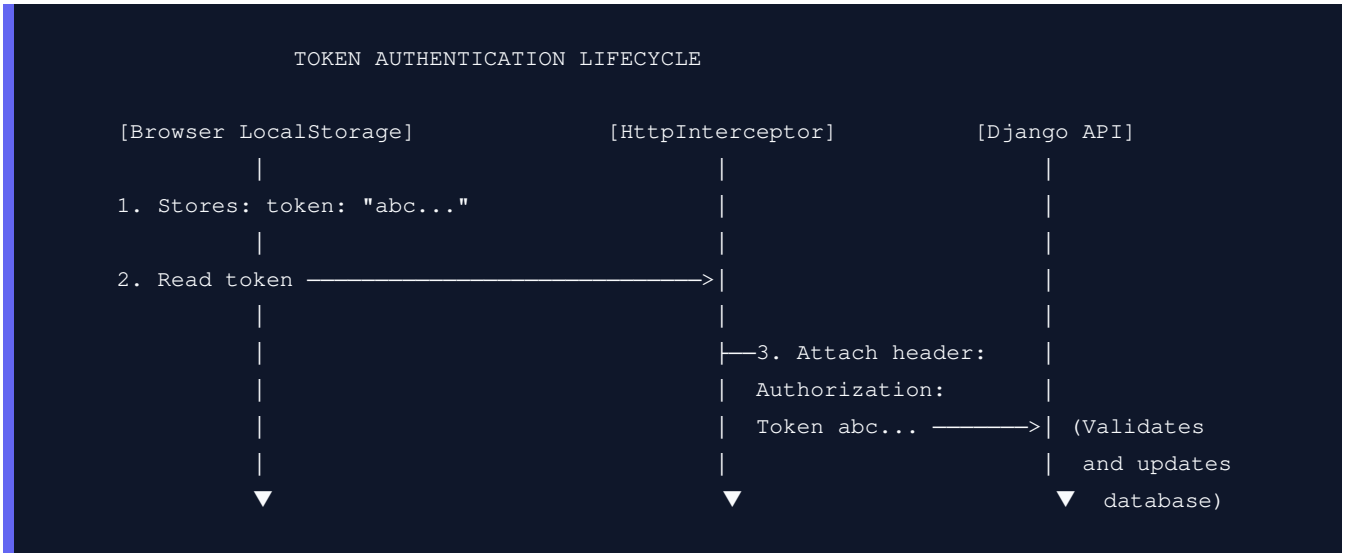
3. Live Coding Walkthrough	23 min	Write interceptor functions, enable HTTP interceptors, write CanActivate guards, and secure edit routes.
4. Check for Understanding	5 min	Q&A on token theft prevention, interceptor order, and bypass strategies.
5. Class Wrap-up & Checkpoint	3 min	Verify unauthorized access is blocked, and valid tokens authorize additions.

🚀 Step-by-Step Walkthrough

1. Hook & Big Picture (7 min)

In Day 4, we configured our Django backend to require Token Authentication for write operations. If you attempt to add a game now via Angular, the backend rejects it, returning a `401 Unauthorized` error. To authorize requests, the user must log in, receive a secure token, store it in the browser, and send that token in the HTTP header of every subsequent request.

- **Security & Auth Lifecycle**: User enters credentials → Angular POSTs to `/register/` or `/login/` → API returns token → Angular saves token in `localStorage` → **HttpInterceptor** intercepts all outbound API requests and attaches the token header → **Route Guard** blocks users from loading pages like `/games/add` if they do not have a token.



2. Key Concepts & Core Ideas (7 min)

Introduce these security mechanisms:

- **Token Authentication**: A stateless auth method. The server issues a cryptographically signed token. The client stores it and attaches it to request headers, eliminating the need for server-side session files.

- : A key-value browser storage mechanism that persists data even after the browser tab is closed.
- **HTTP Interceptor**: A middleware function that intercepts all incoming or outgoing HTTP requests. It is the perfect place to inject headers or handle global error codes (like redirecting to login on 401s) in one central place.
- **Route Guard (`CanActivateFn`)**: A boolean security function linked to route configurations. If the function returns `true`, the navigation completes; if it returns `false` (or redirects), the navigation is cancelled.

---

3. Live Coding Walkthrough (23 min)

Step 3.1: Storing and Retrieving Authentication Tokens

- **Concept**: Create an authentication helper service `AuthService` under `src/app/services/auth.service.ts` to manage token storage and handle login operations.
- **Code**: Generate service:

```
ng generate service services/auth
```

Modify `src/app/services/auth.service.ts`:

```
import { Injectable } from '@angular/core';

@Injectable({
  providedIn: 'root'
})
export class AuthService {
  private tokenKey = 'authToken';

  // Save token in browser storage
  setToken(token: string): void {
    localStorage.setItem(this.tokenKey, token);
  }

  // Retrieve token from browser storage
  getToken(): string | null {
    return localStorage.getItem(this.tokenKey);
  }

  // Delete token (logout)
  logout(): void {
    localStorage.removeItem(this.tokenKey);
  }

  // Check if token exists
  isAuthenticated(): boolean {
    return this.getToken() !== null;
  }
}
```

```
  ]
};
```

Step 3.3: Creating the Route Guard

- **Concept**: Create a functional route guard `authGuard` that checks if the user is authenticated. If they are not, it redirects them to the catalog page.
- **Code**: Generate guard:

```
ng generate guard guards/auth
```

Modify `src/app/guards/auth.guard.ts`:

```
import { CanActivateFn, Router } from '@angular/router';
import { inject } from '@angular/core';
import { AuthService } from '../services/auth.service';

export const authGuard: CanActivateFn = (route, state) => {
  const authService = inject(AuthService);
  const router = inject(Router);

  // Verify token exists in localStorage
  if (authService.isAuthenticated()) {
    return true; // Authorize route activation
  }

  // If unauthenticated, redirect to catalog page
  router.navigate(['/games']);
  return false; // Deny route activation
};
```

Register the guard on the protected routes inside `src/app/app.routes.ts`:

```
import { Routes } from '@angular/router';
import { GameListComponent } from './components/game-list/game-list.component';
import { GameDetailComponent } from './components/game-detail/game-detail.component';
import { AddGameFormComponent } from './components/add-game-form/add-game-form.component';
import { authGuard } from './guards/auth.guard'; // Import route guard

export const routes: Routes = [
  { path: '', redirectTo: 'games', pathMatch: 'full' },
  { path: 'games', component: GameListComponent },
  # Attach canActivate guard: routes to 'games/add' will check authGuard first!
  { path: 'games/add', component: AddGameFormComponent, canActivate: [authGuard] },
  { path: 'games/:id', component: GameDetailComponent },
  { path: '**', redirectTo: 'games' }
];
```

Step 3.4: Simulating Login for Testing

- **Concept**: To quickly test our authentication, let's write a mock login/logout command in `app.component.ts` or log directly in using developer tools console.
- **Code**: Open your browser, navigate to `http://localhost:4200/`. Right-click and select Inspect -> console. To simulate logging in, paste a valid Django auth token in the localStorage (fetch this token from your Django registration/login tests or SQLite database):

```
localStorage.setItem('authToken', 'YOUR_HEX_AUTHENTICATION_TOKEN');
```

- **Verify**:
  - Clear your browser's localStorage using the Application tab or Console: `localStorage.clear()`.
  - Try clicking the "Add Game" link or navigating to `http://localhost:4200/games/add`. Confirm that the guard intercepts you and redirects you back to the catalog list.
  - Now, paste a valid token in the localStorage via the console: `localStorage.setItem('authToken', 'abc123token')`.
  - Click "Add Game". Verify that the form loads successfully. Fill in the form and submit: check the Network tab in your developer tools to verify that the request contains the header: `Authorization: Token abc123token`.

---

4. Check for Understanding (5 min)

> ***Question**: "Why do we clone the HTTP request object inside interceptors instead of editing it directly? e.g. `req.headers.set(...)`?"*
> ***Answer**: In Angular, HTTP request objects are immutable. This is a design pattern that prevents race conditions and makes retry operations safe (if a request is modified midway, retrying it with original parameters is impossible). To modify headers, you must clone the request using `.clone()` and specify the header updates.*

> ***Question**: "What is the difference between `localStorage` and `sessionStorage`?"*
> ***Answer**: `localStorage` persists data permanently across browser sessions; the data remains in the browser even if you close the tab or shut down the computer. `sessionStorage` only persists data for the duration of the page session (as long as that specific browser tab remains open); closing the tab destroys the stored data.*

> ***Question**: "Can route guards protect client-side source code from being downloaded by unauthorized users?"*
> ***Answer**: No. Route guards only control navigation within the running client-side application. Because all Javascript bundles are downloaded to the browser, any user can inspect the source code files.*

> *Sensitive business logic or database queries must always be secured on the server-side backend.*

5. Daily Checkpoint & Wrap-up (3 min)

- **Verification Criteria**:
  - ○ Navigating to `/games/add` without a token in `localStorage` redirects to `/games`.
  - ○ Having a token in `localStorage` permits loading the creation form page.
  - ○ Outbound HTTP requests contain the custom `Authorization` header containing the correct token key value.